SOFTWARE DESIGN DOCUMENT
for
APPOINTMENT MANAGEMENT SYSTEM

1.0

14.12.2025

Prepared by

Bilgenur Türkel

Musab Kardeş

Ömer Faruk Olkay

Rabia Çerçi

Tayyip Soner Tekin

TABLE OF CONTENTS

1 Introduction

This document provides a comprehensive architectural overview of the Appointment Management System. It outlines the system's design goals, subsystem decomposition, and key technical decisions based on the requirements defined in the Requirements Analysis Document (RAD).

1.1 Purpose of the system

The primary purpose of the Appointment Management System is to design and develop a multi-tenant web and mobile platform that allows service based companies to manage their appointments, employees, and resources efficiently. Currently, many businesses use manual methods or disconnected tools, which leads to scheduling conflicts and double-booking issues.

This new system aims to solve these problems by providing a centralized platform where:

- **Customers** can view available services and book appointments online.
- **Branch Managers** can manage services, resources, and employee schedules.
- **Employees** can view their daily schedules and manage appointment requests.
- **Super Admins** can manage the companies and branch managers registered in the system.

1.2 Design goals

The software architecture is designed to meet the specific functional and non-functional requirements defined in the RAD. The key design goals are:

- **Reliability and Availability:** The system must maintain 99.9% uptime during operational hours to ensure companies can always access their schedules.
- **Performance:** The system is designed to handle high traffic. Appointment availability queries must return results in less than 2 seconds, and calendar views must load within 3 seconds.
- **Security (Access Control):** The architecture must enforce **Role-Based Access Control (RBAC)**. This ensures that users (such as Employees or Customers) can only access functions and data appropriate for their role.
- **Scalability (Multi-tenancy):** The system is designed as a multi-tenant solution. This means multiple independent companies can use the platform while keeping their data completely separate.
- **Maintainability:** The project follows an **API-first approach**. The backend is built with **Spring Boot (Java)** for robust API services, and the frontend uses **React** to create a Progressive Web App (PWA)

1.3 Definitions, acronyms, and abbreviations

The following terms and abbreviations are used throughout this document:

- **API (Application Programming Interface):** A set of rules that allows different software entities to communicate with each other.
- **CRUD:** An acronym for Create, Read, Update, and Delete operations. These are the basic operations for managing data.
- **Customer:** The end-user who books an appointment.
- **Employee:** The staff member who provides the 0service for the appointment.

- **Branch Manager:** The user who manages employees, services, and resources for a company.
- **Super Admin:** The highest-level administrator, who manages the entire system and all businesses.
- **Multi-Tenant:** An architecture where a single instance of the software serves multiple independent organizations (tenants).
- **PWA (Progressive Web App):** A web application that provides a user experience similar to a native mobile app.
- **RAD:** Requirements Analysis Document.
- **RBAC:** Role-Based Access Control.
- **SDD:** Software Design Document.
- **SRS:** Software Requirements Specification.
- **RDBMS:** Relational Database Management System

1.4 References

- Object-Oriented Software Engineering, Using UML, Patterns, and Java, 3rd Edition, by Bernd Bruegge and Allen H. Dutoit, Prentice-Hall, 2010, ISBN 10: 0136066836.

- Headfirst Design Patterns: Building Extensible and Maintainable Object Oriented Software, 2nd Edition, O'Reilly Media, 2020. introduction.

1.5 Overview

This document describes the software design and architecture of the Appointment Management System. The structure of the document is as follows:

- **Section 2** describes the current situation and the architecture of the systems being replaced (manual systems and disconnected tools).
- **Section 3** details the **Proposed Software Architecture**, including the subsystem decomposition, hardware/software mapping, persistent data management, and access control mechanisms.
- **Section 4** defines the services provided by each subsystem.
- **Section 5** provides a glossary of terms used in the design.
- **Section 6** ensures traceability between the design components and the requirements defined in the RAD.

2 Current software architecture

Since this project aims to build a new multi-tenant platform, there is no single legacy software system being replaced. Instead, the current "architecture" used by most target companies consists of a mix of manual methods and disconnected digital tools.

This section surveys these existing methods to highlight the background information and common issues that the new **Appointment Management System** will address.

2.1 Existing Methods

Currently, service-based companies typically manage their operations using one of the following approaches:

- **Manual Systems:** Many businesses still rely on pen-and-paper logging and phone calls to schedule appointments.

- **Disconnected Digital Tools:** Some companies use general-purpose software such as spreadsheets (e.g., Excel) or standalone calendar applications (e.g., Google Calendar).

2.2 Architectural Issues

The primary limitation of the current model is the lack of a centrally coordinated platform. Because these tools are not connected, data is scattered across different places. This "disconnected architecture" leads to several critical issues:

- **Scheduling Conflicts:** Without a central system checking availability, "double-booking" of employees or equipment frequently occurs.
- **Inefficient Resource Allocation:** Managers cannot easily see which resources are in use or available, leading to poor management of equipment and staff.
- **Communication Gaps:** There is no automated way to send confirmations or reminders, which can result in poor communication with customers.

The new system assumes that replacing these disconnected tools with a single, centralized platform will solve these coordination problems and improve operational efficiency.

3 Proposed software architecture

3.1 Overview

The proposed system follows a **Client-Server architecture** utilizing a layered approach to ensure separation of concerns, scalability, and maintainability.

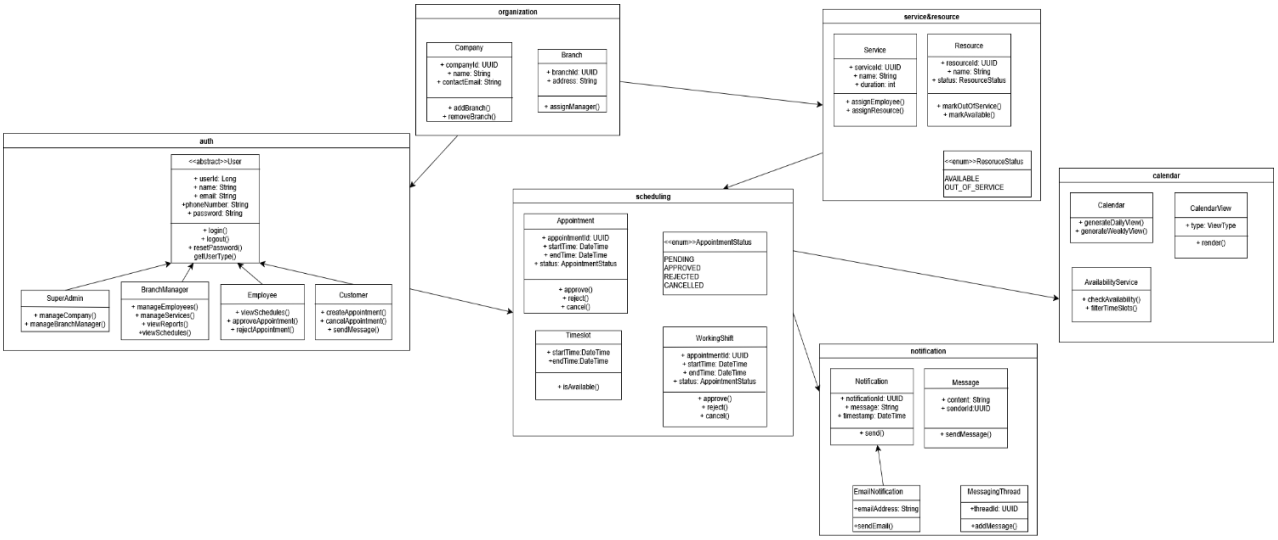
- **Frontend (Client):** Developed as a **Progressive Web App (PWA)** using **React**. It provides a responsive user interface for Customers, Employees, and Managers.
- **Backend (Server):** Built with **Spring Boot (Java)**, exposing a **RESTful API**. It handles business logic, security, and database interactions.
- **Communication:** The client and server communicate via **HTTP/HTTPS** using **JSON** as the data interchange format.
- **Database:** A relational database (RDBMS) PostgreSQL is used to store persistent data such as users, appointments, and company information.

3.2 Subsystem decomposition

The system is decomposed into logical subsystems based on their responsibilities. This decomposition minimizes dependencies and promotes modular development.

- **Authentication Subsystem:** Handles user registration, login, password reset, and token management (JWT). It ensures that only authenticated users can access the system.
- **Scheduling Subsystem:** The core engine responsible for managing availability, booking appointments, and preventing double-booking conflicts. It filters time slots based on service duration and resource availability.
- **User Management Subsystem:** Manages profiles for all roles (Super Admin, Manager, Employee, Customer) and handles Role-Based Access Control (RBAC).
- **Resource & Service Management Subsystem:** Allows Branch Managers to define services, assign employees to services, and manage physical resources (e.g., rooms, equipment).

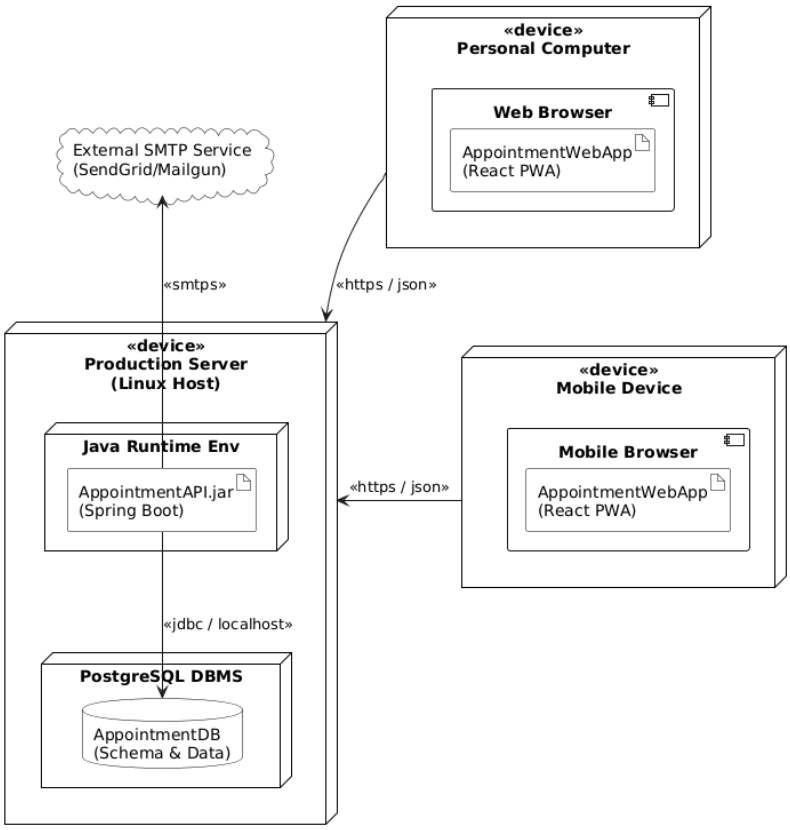
- **Notification Subsystem:** Manages the generation and sending of transactional emails (confirmations, cancellations, reminders).



3.3 Hardware/software mapping

The following descriptions detail the mapping of software components to the hardware nodes as depicted in the Deployment Diagram.

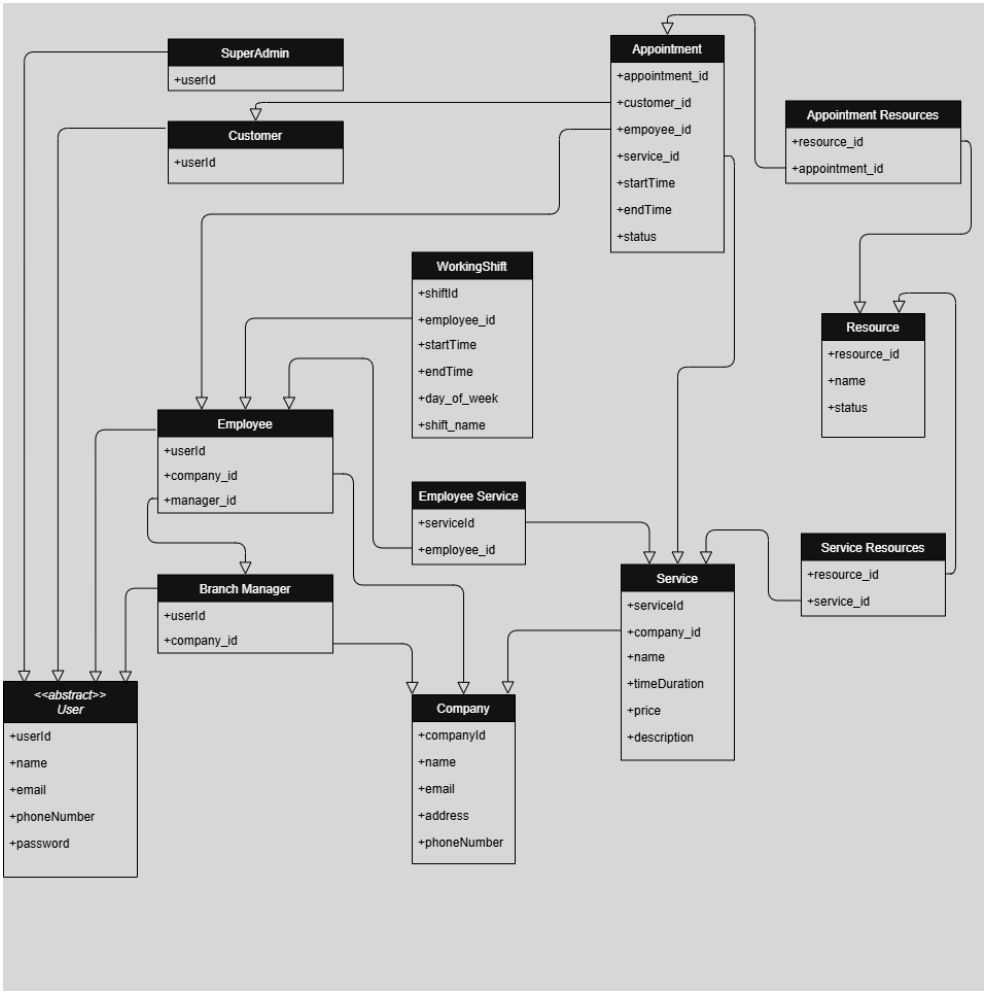
- **Production Server:** A centralized Linux-based server that hosts both the application and database layers to simplify deployment. It contains the **Java Runtime Environment (JRE)** to execute the Spring Boot backend (AppointmentAPI.jar) and the **PostgreSQL DBMS** to manage the AppointmentDB schema. Communication between the API and the database occurs internally via JDBC over localhost.
- **Personal Computer:** Represents the end-user's desktop or laptop environment. It runs a standard **Web Browser** which hosts the client-side application (AppointmentWebApp), implemented as a React Progressive Web App (PWA). It communicates with the Production Server via secure HTTPS requests exchanging JSON data.
- **Mobile Device:** Represents portable devices such as smartphones or tablets used by customers or employees on the go. It utilizes a **Mobile Browser** to execute the same AppointmentWebApp (React PWA), ensuring a consistent cross-platform experience. Like the Personal Computer, it connects to the Production Server over HTTPS.



3.4 Persistent data management

The system uses a **Relational Database Management System (RDBMS)** to ensure data integrity and support complex queries required for scheduling.

- **Data Model:** Key entities include User, Company, Service, Appointment, Resource, and WorkingShift
- **Data Access:** The backend uses an **Object-Relational Mapping (ORM)** framework (Hibernate/JPA) to interact with the database, abstracting SQL queries.
- **Multi-tenancy:** Data isolation is enforced at the application level; every query filters data by company_id to ensure tenants cannot access each other's data.


SuperAdmin
+userId
Customer
+userId
Appointment
+appointment_id
+customer_id
+empoyee_id
+service_id
+startTime
+endTime
+status
Appointment Resources
+resource_id
+appointment_id
WorkingShift
+shiftId
+employee_id
+startTime
+endTime
+day_of_week
+shift_name
Resource
+resource_id
+name
+status
Employee
+userId
+company_id
+manager_id
Employee Service
+serviceId
+employee_id
Service Resources
+resource_id
+service_id
Branch Manager
+userId
+company_id
Service
+serviceId
+company_id
+name
+timeDuration
+price
+description
<<abstract>>
User
+userId
+name
+email
+phoneNumber
+password
Company
+companyId
+name
+email
+address
+phoneNumber

3.5 Access control and security

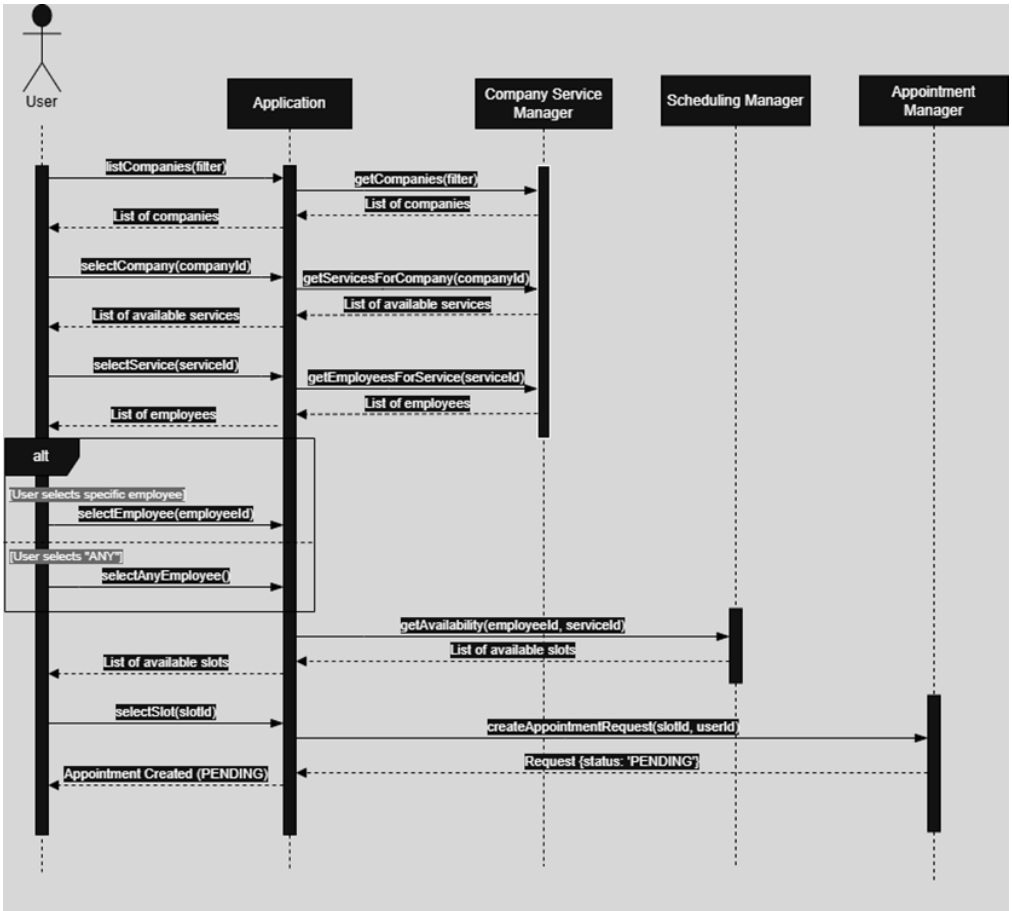
- **Authentication:** The system uses **JSON Web Tokens (JWT)** for stateless authentication. Upon successful login, the server issues a token which the client must include in the header of subsequent requests.
- **Authorization: Role-Based Access Control (RBAC)** is implemented.
- *Super Admin:* Can manage companies.
- *Branch Manager:* Can manage services, employees, and resources.
- *Employee:* Can view schedules and manage requests.
- *Customer:* Can book and manage their own appointments.
- **Data Security:** Passwords are hashed before storage using BCrypt algorithm. All data transmission occurs over encrypted channels (HTTPS).

3.6 Global software control

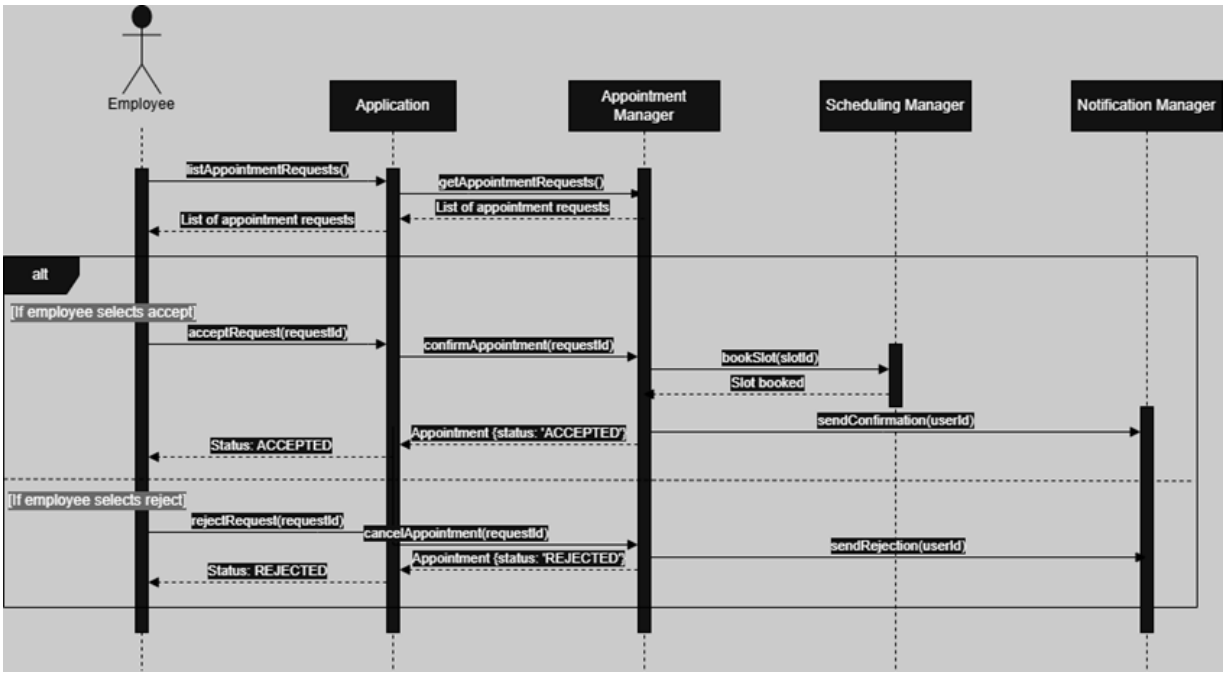
The control flow is primarily **Request-Response** driven, initiated by user actions on the client side.

- **Synchronous Operations:** Most operations (e.g., viewing the calendar, booking a slot) are synchronous API calls where the client waits for the server's response.
- **Asynchronous Operations:** Non-blocking tasks, such as sending email notifications, are handled asynchronously. The AppointmentService triggers an event to the NotificationService without making the user wait for the email delivery to complete.

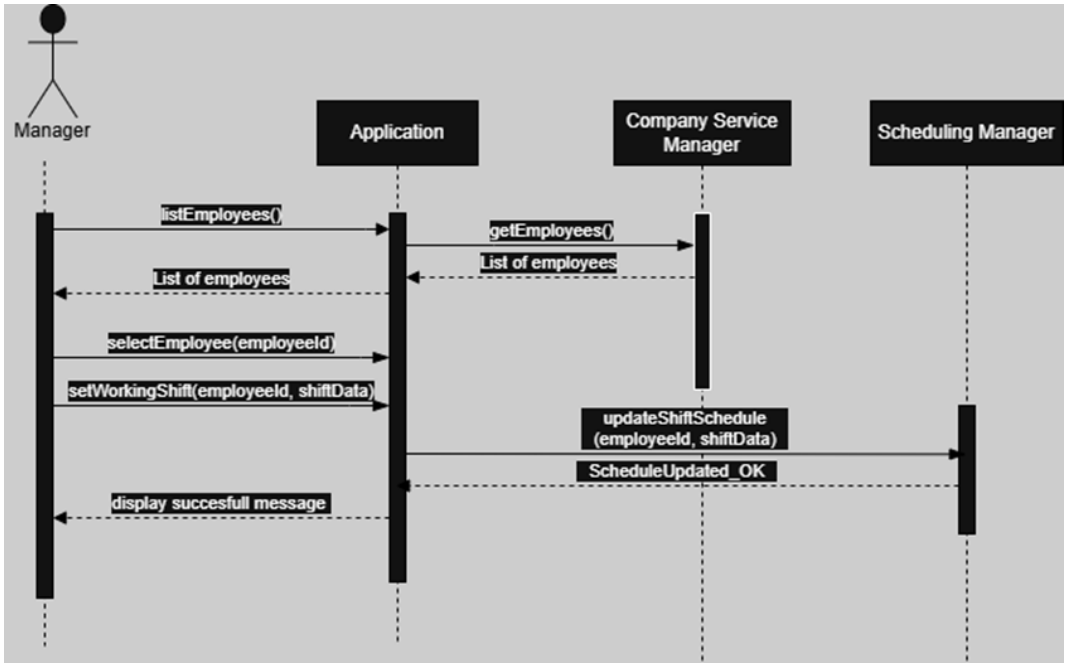
Create an Appointment Request



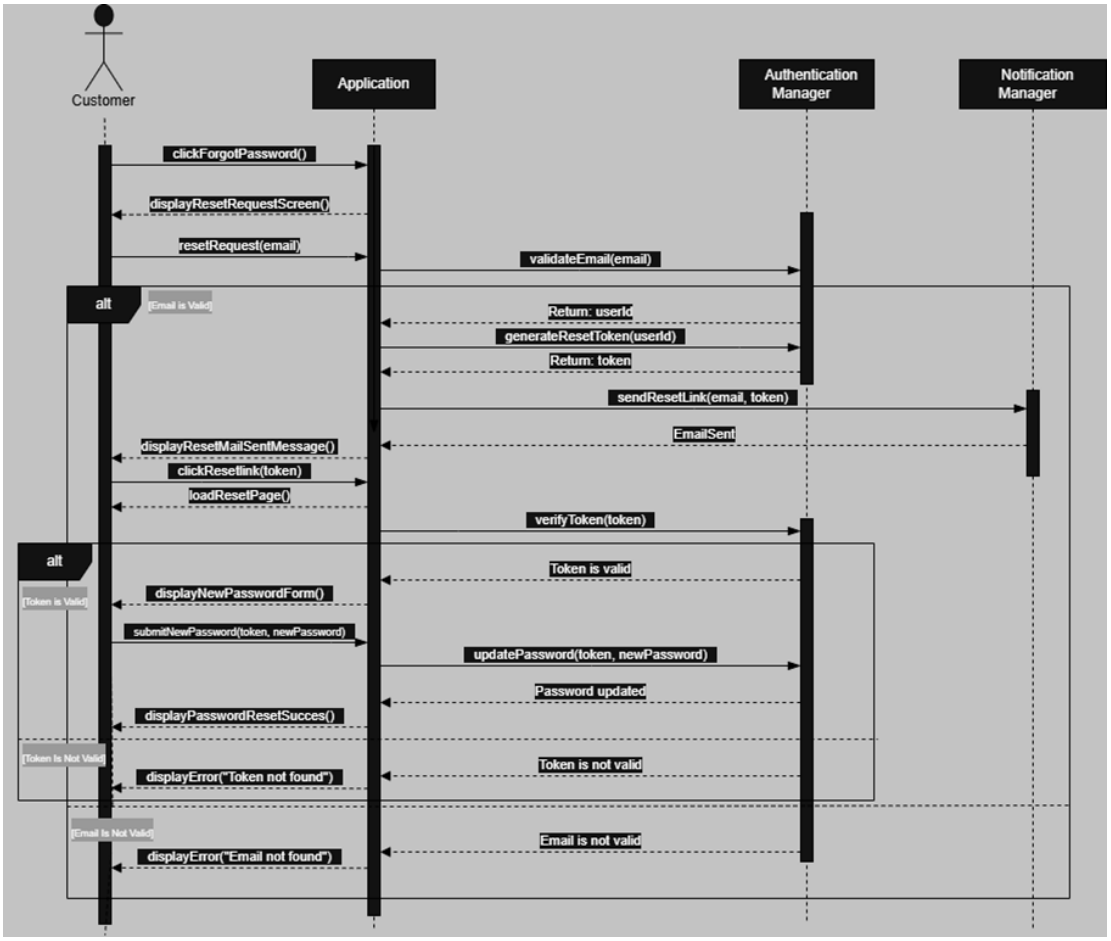
Handle Appointment Request



Set Employee Schedule



Reset Password



3.7 Boundary conditions

- **Start-up:** The backend application validates database connectivity and loads necessary configurations upon initialization.
- **Shutdown:** Graceful shutdown procedures ensure that active database connections are closed properly to prevent data corruption.
- **Error Handling:** Global exception handlers capture system errors. User-friendly error messages are returned to the client (e.g., "Slot unavailable"), while detailed stack traces are logged internally for debugging

4 Subsystem services

This section outlines the services provided by each subsystem identified in the proposed architecture. These services define the interface and operations available to other subsystems or external clients.

4.1 Authentication Subsystem

This subsystem manages user identity and security tokens.

1. **Login(email, password):** Verifies user credentials and issues a JWT access token.

2. **Register(userData):** Creates a new user account (specifically for Customers).
3. **ResetPassword(email):** Initiates the password reset process via email token.
4. **ValidateToken(token):** Checks if the provided JWT is valid and active.

4.2 Scheduling Subsystem

This subsystem handles the core logic for appointments and availability.

1. **GetAvailability(serviceId, employeeId, date):** Returns a list of available time slots based on the service duration and employee shift.
2. **CreateAppointmentRequest(appointmentData):** Creates a new appointment with "PENDING" status.
3. **ApproveAppointment(appointmentId):** Changes the appointment status to "APPROVED" and triggers a notification.
4. **RejectAppointment(appointmentId):** Changes the status to "REJECTED".
5. **CancelAppointment(appointmentId):** Cancels an existing appointment and frees up the time slot.

4.3 User Management Subsystem

This subsystem handles the profiles and roles of the system users.

1. **CreateCompany(companyData):** Adds a new company tenant to the system (Super Admin only).
2. **CreateEmployee(employeeData):** Adds a new employee to a specific branch.
3. **SetWorkingShift(employeeId, shiftData):** Defines the start and end times for an employee's schedule.
4. **GetEmployeeProfile(employeeId):** Retrieves detailed information about a specific employee.

4.4 Resource & Service Management Subsystem

This subsystem manages the assets and service definitions of a branch.

1. **CreateService(serviceData):** Defines a new service with a specific duration and description.
2. **AssignServiceToEmployee(serviceId, employeeId):** Links a specific skill/service to an employee.
3. **CreateResource(resourceData):** Adds a physical resource (e.g., room, equipment) to the inventory.
4. **SetResourceStatus(resourceId, status):** Marks a resource as "Available" or "Out of Service".

4.5 Notification Subsystem

This subsystem handles asynchronous communication.

1. **SendConfirmationEmail(userId, appointmentDetails):** Sends a booking confirmation email
2. **SendCancellationEmail(userId, appointmentDetails):** Notifies the user that an appointment has been cancelled.
3. **SendReminderEmail(userId, appointmentDetails):** Sends a reminder before the scheduled time.

5 Glossary of Terms

Appointment Status: Defines the current state of a booking within its lifecycle. Key states include:

- **PENDING:** The appointment is requested by a customer but not yet confirmed by the employee or manager.
- **APPROVED:** The appointment is confirmed and scheduled on the calendar.
- **REJECTED:** The appointment request was denied by the staff.
- **CANCELLED:** The appointment was revoked by the customer or staff after initial approval.

API-First Approach: A design strategy where the Application Programming Interface (API) is designed and implemented before the user interface. This allows the backend (Spring Boot) and frontend (React) to be developed in parallel.

JWT (JSON Web Token): A compact, URL-safe means of representing claims to be transferred between two parties. In this system, it is used to securely transmit user identity between the React frontend and Spring Boot backend without server-side sessions.

Double-Booking: A scheduling conflict where two different appointments are assigned to the same employee or resource at the same overlapping time slot. The system's Scheduling Subsystem is designed to prevent this.

Multi-Tenancy: A software architecture in which a single instance of the software runs on a server and serves multiple distinct user groups (tenants). In this system, each "Company" is a tenant, and their data is logically isolated using a `company_id` filter in the database.

Resource: A physical asset or facility required to perform a service (e.g., a massage room, a dental chair, or specific equipment). Resources are managed by Branch Managers and must be checked for availability alongside employees.

Service: A specific type of work or treatment offered by the company (e.g., "Haircut," "Dental Cleaning") that has a defined duration and price. Services are assigned to employees who have the skill to perform them.

Stateless Authentication: A security mechanism where the server does not store session information for logged-in users. Instead, the client sends a JSON Web Token (JWT) with every request to prove identity.

Subsystem: A logical component of the system responsible for a specific set of functionalities (e.g., Scheduling Subsystem, Notification Subsystem). This decomposition helps in minimizing dependencies.

Tenant: In the context of this system, a "Tenant" refers to a distinct Company that subscribes to the platform. Each tenant has its own isolated list of employees, services, and customers.

Working Shift: The defined time period during a day when an employee is available to accept appointments. This is used by the logic to calculate available time slots.

6 Traceability

Req ID	Requirement Description	Design Subsystem	Implementation Component / Service
R-01	User Authentication: Users (Admins, Managers, Employees, Customers) must be able to log in securely.	Authentication Subsystem	4.1.1 Login(email, password) 4.1.4 ValidateToken(token)
R-02	Account Creation: New customers must be able to register an account.	Authentication Subsystem	4.1.2 Register(userData)
R-03	Password Recovery: Users must be able to reset forgotten passwords via email.	Authentication Subsystem	4.1.3 ResetPassword(email)
R-04	View Availability: Customers must see available time slots based on service duration.	Scheduling Subsystem	4.2.1 GetAvailability(serviceId, employeeId, date)
R-05	Book Appointment: Customers can request a new appointment.	Scheduling Subsystem	4.2.2 CreateAppointmentRequest(appointmentData)
R-06	Conflict Prevention: The system must prevent double-booking of employees or resources.	Scheduling Subsystem	4.2.1 GetAvailability(serviceId, employeeId, date) 4.2.2 CreateAppointmentRequest(appointmentData)
R-07	Manage Appointments: Employees must be able to approve or reject pending requests.	Scheduling Subsystem	4.2.3 ApproveAppointment(appointmentId) 4.2.4 RejectAppointment(appointmentId)
R-08	Employee Scheduling: Managers must define working shifts for employees.	User Management Subsystem	4.3.3 SetWorkingShift(employeeId, shiftData)
R-9	Service Management: Managers must define services and assign them to employees.	Resource & Service Management Subsystem	4.4.1 CreateService(serviceData) 4.4.2 AssignServiceToEmployee(...)
R-10	Resource Management: Managers must track physical assets (rooms/equipment).	Resource & Service Management Subsystem	4.4.3 CreateResource(resourceData) 4.4.4 SetResourceStatus(...)
R-11	Email Notifications: Users receive automated confirmations and cancellations.	Notification Subsystem	4.5.1 SendConfirmationEmail(...) 4.5.2 SendCancellationEmail(...)

Req ID	NFR Description	Architectural Decision
NFR-01	Performance: Availability queries must return in < 2 seconds.	Database Indexing & Efficient SQL in Scheduling Subsystem
NFR-02	Security: Access must be restricted based on user roles.	RBAC Implementation in User Management Subsystem
NFR-03	Scalability: System must support multiple independent companies.	Multi-tenant Database Architecture (Shared Schema / Isolated Data)
NFR-04	Accessibility: System must be accessible via mobile and desktop.	Frontend developed as a Progressive Web App (PWA) in React